

Convergence and Generalization in Neural Networks

Neural Tangent Kernel

NIPS 2018

김동희

Fundamental Team

Introduction

수식 폭탄

The *kernel gradient* $\nabla_K C|_{f_0} \in \mathcal{F}$ is defined as $\Phi_K \left(\partial_t^n C|_{f_0} \right)$. In contrast to $\partial_t^n C$ which is only defined on the dataset, the kernel gradient generalizes to values x outside the dataset thanks to the kernel K :

$$\nabla_K C|_{f_0}(x) = \frac{1}{N} \sum_{j=1}^N K(x, x_j) d|_{f_0}(x_j).$$

A time-dependent function $f(t)$ follows the *kernel gradient descent with respect to K* if it satisfies the differential equation

$$\partial_t f(t) = -\nabla_K C|_{f(t)}.$$

During kernel gradient descent, the cost $C(f(t))$ evolves as

$$\partial_t C|_{f(t)} = -\langle d|_{f(t)}, \nabla_K C|_{f(t)} \rangle_{p^n} = -\|d|_{f(t)}\|_K^2.$$

Convergence to a critical point of C is hence guaranteed if the kernel K is positive definite with respect to $\|\cdot\|_{p^n}$: the cost is then strictly decreasing except at points such that $\|d|_{f(t)}\|_{p^n} = 0$. If the cost is convex and bounded from below, the function $f(t)$ therefore converges to a global minimum as $t \rightarrow \infty$.

3.1 Random functions approximation

As a starting point to understand the convergence of ANN gradient descent to kernel gradient descent in the infinite-width limit, we introduce a simple model, inspired by the approach of [15].

A kernel K can be approximated by a choice of P random functions $f^{(p)}$ sampled independently from any distribution on $\mathcal{F}$ whose (non-centered) covariance is given by the kernel K :

$$\mathbb{E}[f_k^{(p)}(x)f_k^{(p)}(x')] = K_{kk}(x, x').$$

These functions define a random linear parametrization $F^{lin} : \mathbb{R}^P \rightarrow \mathcal{F}$

$$\theta \mapsto f_\theta^{lin} = \frac{1}{\sqrt{P}} \sum_{p=1}^P \theta_p f^{(p)}.$$

The partial derivatives of the parametrization are given by

$$\partial_{\theta_p} F^{lin}(\theta) = \frac{1}{\sqrt{P}} f^{(p)}.$$

Optimizing the cost $C \circ F^{lin}$ through gradient descent, the parameters follow the ODE:

$$\partial_t \theta_p(t) = -\partial_{\theta_p} (C \circ F^{lin})(\theta(t)) = -\frac{1}{\sqrt{P}} \partial_t^n C|_{f_\theta^{lin}}, \quad f^{(p)} = -\frac{1}{\sqrt{P}} \langle d|_{f_\theta^{lin}}, f^{(p)} \rangle_{p^n}.$$

As a result the function f_θ^{lin} evolves according to

$$\partial_t f_\theta^{lin} = \frac{1}{\sqrt{P}} \sum_{p=1}^P \partial_t \theta_p(t) f^{(p)} = -\frac{1}{P} \sum_{p=1}^P \langle d|_{f_\theta^{lin}}, f^{(p)} \rangle_{p^n} f^{(p)},$$

where the right-hand side is equal to the kernel gradient $-\nabla_K C$ with respect to the *tangent kernel*

$$\tilde{K} = \sum_{p=1}^P \partial_{\theta_p} F^{lin}(\theta) \otimes \partial_{\theta_p} F^{lin}(\theta) = \frac{1}{P} \sum_{p=1}^P f^{(p)} \otimes f^{(p)}.$$

This is a random n_L -dimensional kernel with values $\tilde{K}_{ii'}(x, x') = \frac{1}{P} \sum_{p=1}^P f_i^{(p)}(x) f_{i'}^{(p)}(x')$.

Performing kernel gradient descent on the cost $C \circ F^{lin}$ is therefore equivalent to performing kernel gradient descent with the tangent kernel $\tilde{K}$ in the function space. In the limit as $P \rightarrow \infty$, by the law of large numbers, the (random) tangent kernel $\tilde{K}$ tends to the fixed kernel K , which makes this method an approximation of kernel gradient descent with respect to the limiting kernel K .

4 Neural tangent kernel

For ANNs trained using gradient descent on the composition $C \circ F^{(L)}$, the situation is very similar to that studied in the Section 3.1. During training, the network function f_θ evolves along the (negative) kernel gradient

$$\partial_t f_\theta(t) = -\nabla_{\Theta^{(L)}} C|_{f_\theta(t)}$$

with respect to the *neural tangent kernel* (NTK)

$$\Theta^{(L)}(\theta) = \sum_{p=1}^P \partial_{\theta_p} F^{(L)}(\theta) \otimes \partial_{\theta_p} F^{(L)}(\theta).$$

However, in contrast to F^{lin} , the realization function $F^{(L)}$ of ANNs is not linear. As a consequence, the derivatives $\partial_{\theta_p} F^{(L)}(\theta)$ and the neural tangent kernel depend on the parameters θ . The NTK is therefore random at initialization and varies during training, which makes the analysis of the convergence of f_θ more delicate.

In the next subsections, we show that, in the infinite-width limit, the NTK becomes deterministic at initialization and stays constant during training. Since f_θ at initialization is Gaussian in the limit, the asymptotic behavior of f_θ during training can be explicated in the function space $\mathcal{F}$.

4.1 Initialization

As observed in [12; 9], the output functions $f_{\theta,i}$ for $i = 1, \dots, n_L$ tend to iid Gaussian processes in the infinite-width limit (a proof in our setup is given in the appendix):

Proposition 1. *For a network of depth L at initialization, with a Lipschitz nonlinearity σ , and in the limit as $n_1, \dots, n_{L-1} \rightarrow \infty$, the output functions $f_{\theta,k}$, for $k = 1, \dots, n_L$, tend (in law) to iid centered Gaussian processes of covariance $\Sigma^{(L)}$, where $\Sigma^{(L)}$ is defined recursively by:*

$$\Sigma^{(1)}(x, x') = \frac{1}{n_0} x^T x' + \beta^2,$$

$$\Sigma^{(L+1)}(x, x') = \mathbb{E}_{f \sim \mathcal{N}(0, \Sigma^{(L)})} [\sigma(f(x)) \sigma(f(x'))] + \beta^2,$$

taking the expectation with respect to a centered Gaussian process f of covariance $\Sigma^{(L)}$.

Remark 2. *Strictly speaking, the existence of a suitable Gaussian measure with covariance $\Sigma^{(L)}$ is not needed: we only deal with the values of f at x, x' (the joint measure on $f(x), f(x')$ is simply a Gaussian vector in $2D$). For the same reasons, in the proof of Proposition 1 and Theorem 1, we will freely speak of Gaussian processes without discussing their existence.*

The first key result of our paper (proven in the appendix) is the following: in the same limit, the Neural Tangent Kernel (NTK) converges in probability to an explicit deterministic limit.

Theorem 1. *For a network of depth L at initialization, with a Lipschitz nonlinearity σ , and in the limit as the layers width $n_1, \dots, n_{L-1} \rightarrow \infty$, the NTK $\Theta^{(L)}$ converges in probability to a deterministic limiting kernel:*

$$\Theta^{(L)} \rightarrow \Theta_\infty^{(L)} \otimes Id_{n_L}.$$

The scalar kernel $\Theta_\infty^{(L)} : \mathbb{R}^{n_0} \times \mathbb{R}^{n_0} \rightarrow \mathbb{R}$ is defined recursively by

$$\Theta_\infty^{(1)}(x, x') = \Sigma^{(1)}(x, x')$$

$$\Theta_\infty^{(L+1)}(x, x') = \Theta_\infty^{(L)}(x, x') \dot{\Sigma}^{(L+1)}(x, x') + \Sigma^{(L+1)}(x, x'),$$

where

$$\dot{\Sigma}^{(L+1)}(x, x') = \mathbb{E}_{f \sim \mathcal{N}(0, \Sigma^{(L)})} [\dot{\sigma}(f(x)) \dot{\sigma}(f(x'))],$$

taking the expectation with respect to a centered Gaussian process f of covariance $\Sigma^{(L)}$, and where $\dot{\sigma}$ denotes the derivative of σ .

Remark 3. *By Rademacher's theorem, $\dot{\sigma}$ is defined everywhere, except perhaps on a set of zero Lebesgue measure.*

Note that the limiting $\Theta_\infty^{(L)}$ only depends on the choice of σ , the depth of the network and the variance of the parameters at initialization (which is equal to 1 in our setting).

Introduction

수식 폭탄

Reviewer 2

Summary: The authors show that an artificial neural network (ANN) with infinite width can be interpreted using Neural Tangent Kernels (NTK). Using the NTK, they show that ANN converges faster on a few large principal components of the NTK which they argue is a motivation for early stopping. Quality: Quite frankly I had a hard time understanding all the mathematical definitions related to kernels. I cannot do better than make an educated guess. Clarity: The motivation, theoretical explanations and experiments were well explained in general. Unfortunately I could not understand it in details. Originality: I am not familiar enough with the subject to evaluate the originality. Significance: The final results and assumptions related to skewed distribution of eigenvalues of the input data with respect to the NTK seems related to similar observations made with respect to the Hessian [1]. Due to my lack of understanding of the theory behind the NTK I cannot tell whether those are truly related or not. [1] Karakida, Ryo, Shotaro Akaho, and Shun-ichi Amari. "Universal Statistics of Fisher Information in Deep Neural Networks: Mean Field Approach." arXiv preprint arXiv:1806.01316 (2018).

이렇게 얕전하고 겸손한 Reviewer 2가...

Introduction

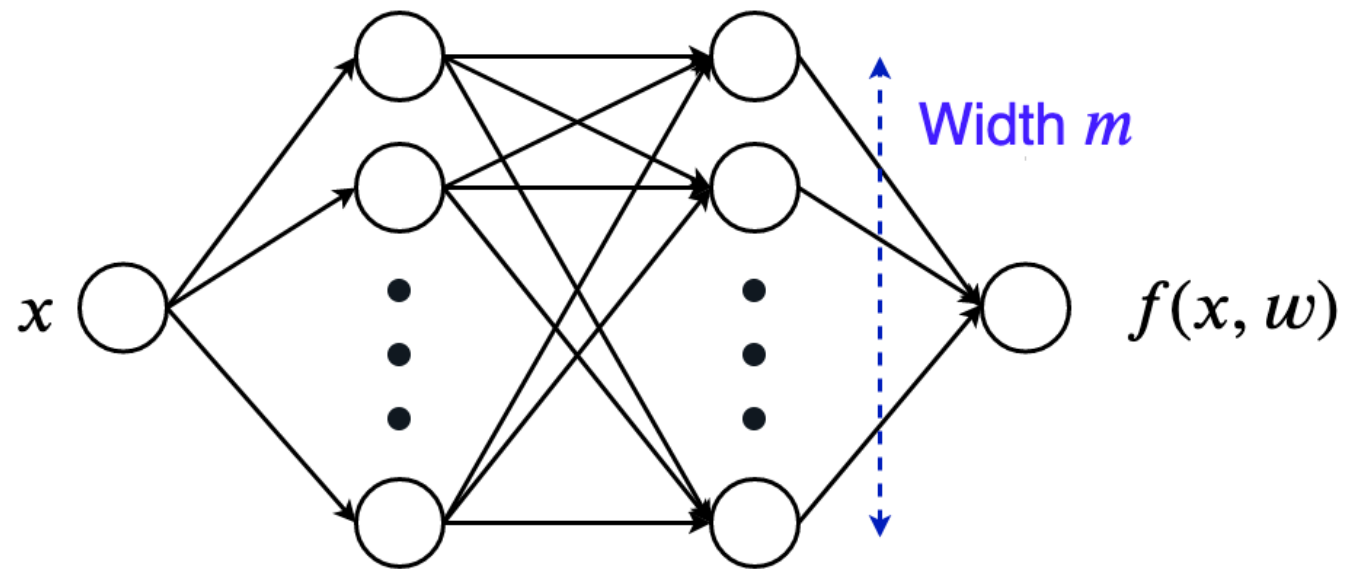
본 발표에서는

논문 내용을 그대로 따라가지 않고

최대한 직관적으로 해석할 수 있도록 설명

Introduction

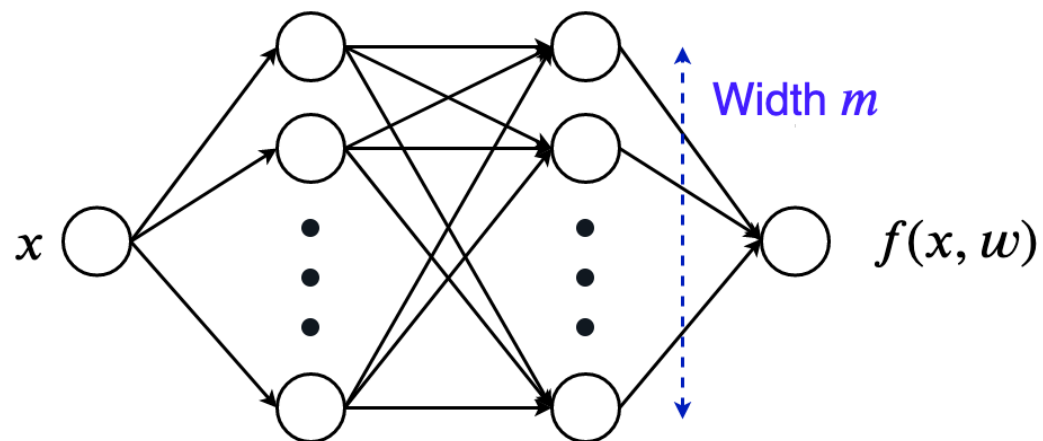
결론



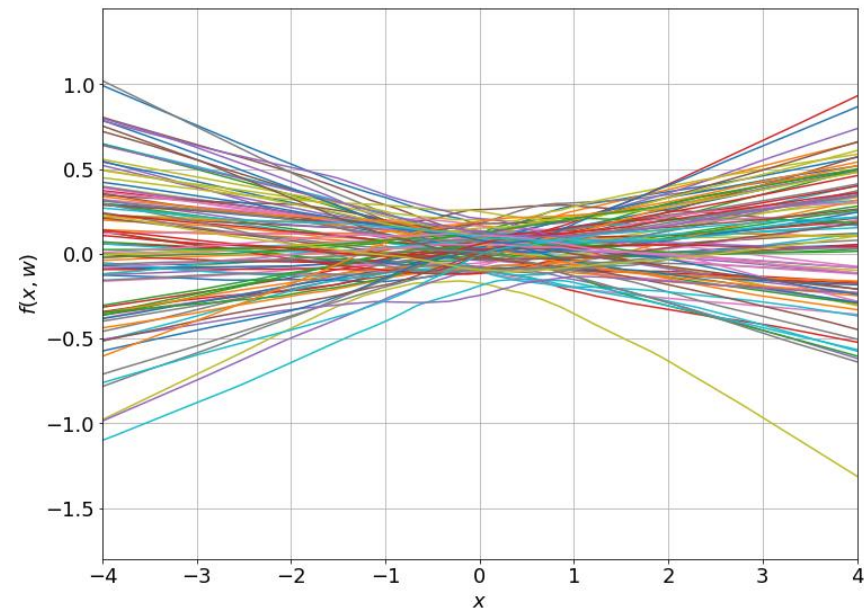
Width가 infinite 한 neural network는 NTK를 사용하면 linear model로 근사할 수 있다!

Introduction

빌드업



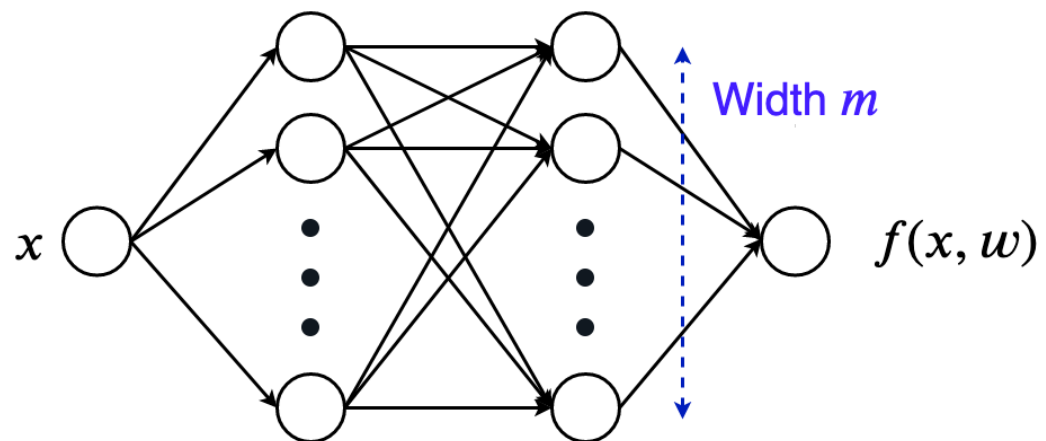
$$L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \frac{1}{2} (f(\bar{x}_i, \mathbf{w}) - \bar{y}_i)^2$$



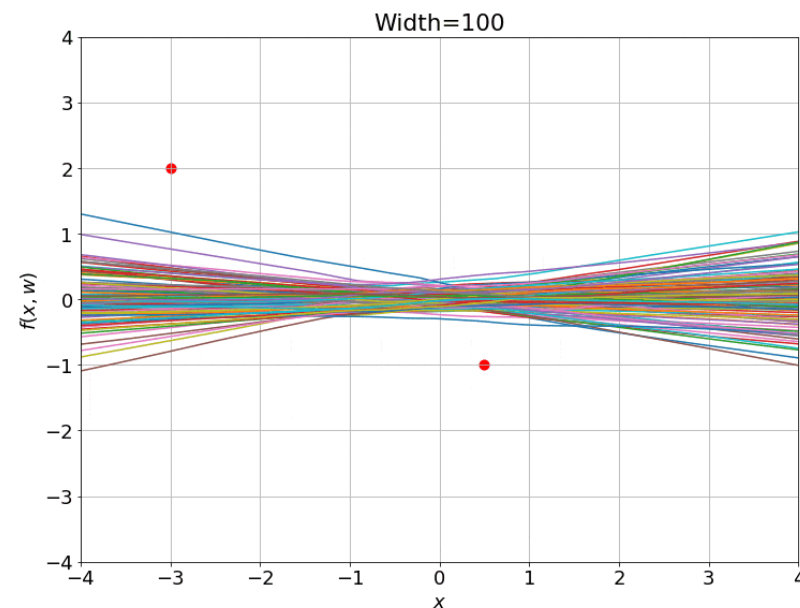
$$L(\mathbf{w}) = \frac{1}{2} \|\mathbf{y}(\mathbf{w}) - \bar{\mathbf{y}}\|_2^2$$

Introduction

빌드업



$$L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \frac{1}{2} (f(\bar{x}_i, \mathbf{w}) - \bar{y}_i)^2$$

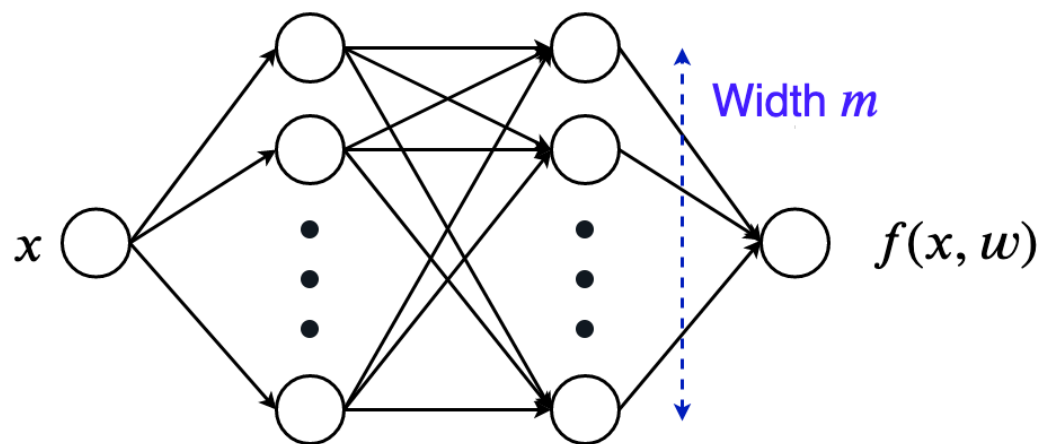


$$L(\mathbf{w}) = \frac{1}{2} \|\mathbf{y}(\mathbf{w}) - \bar{\mathbf{y}}\|_2^2$$

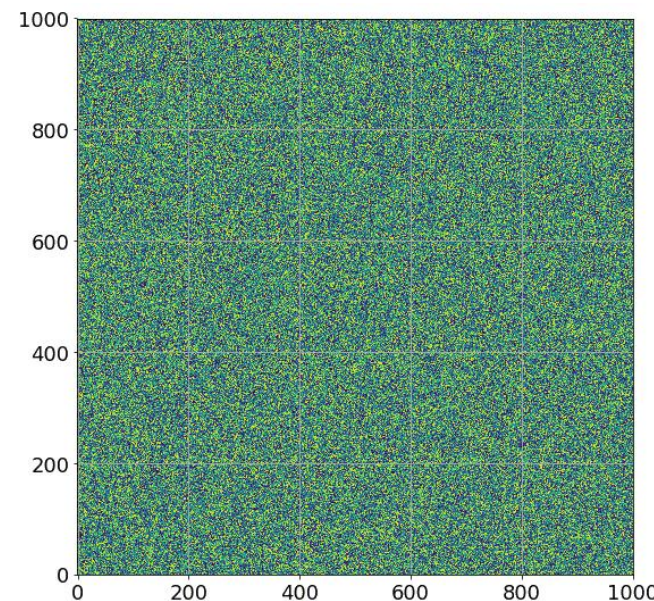
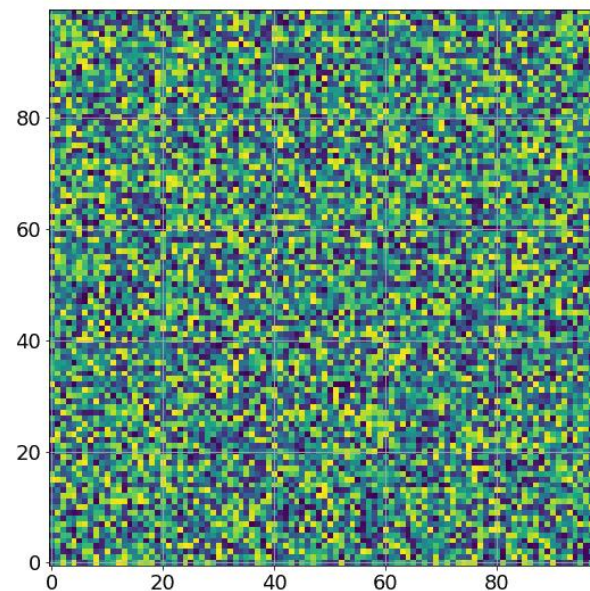
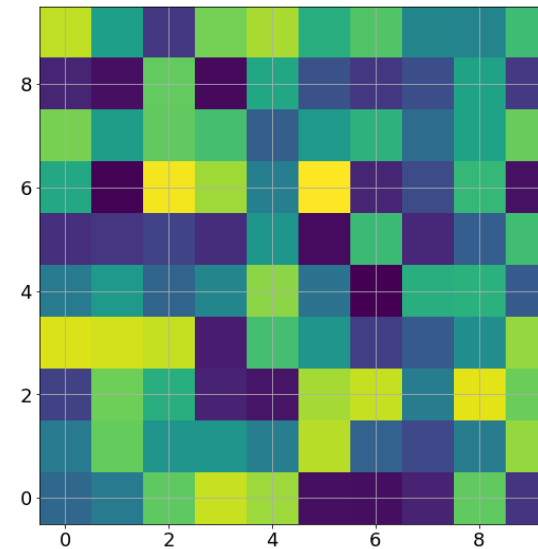
Introduction

빌드업

$$L(\boldsymbol{w}) = \frac{1}{2} \|\boldsymbol{y}(\boldsymbol{w}) - \bar{\boldsymbol{y}}\|_2^2$$



Width가 infinite 한 neural network는
NTK를 사용하면 linear model로 근사할 수 있다!



Q & A

Taylor Expansion

이론편

$$f(x, \mathbf{w}) \approx f(x, \mathbf{w}_0) + \nabla_{\mathbf{w}} f(x, \mathbf{w}_0)^T (\mathbf{w} - \mathbf{w}_0)$$

$$\mathbf{y}(\mathbf{w}) \approx \mathbf{y}(\mathbf{w}_0) + \nabla_{\mathbf{w}} \mathbf{y}(\mathbf{w}_0)^T (\mathbf{w} - \mathbf{w}_0)$$

$n = \# \text{ datapoints}$

$p = \# \text{ parameters}$

$$\nabla_{\mathbf{w}} \mathbf{y}(\mathbf{w}_0) \sim p \times n$$

$$\nabla_{\mathbf{w}} \mathbf{y}(\mathbf{w}_0)^T \sim n \times p$$

$$\begin{array}{c} n \\ \updownarrow \\ \left[\begin{array}{c} \mathbf{y}(\mathbf{w}) \end{array} \right] \\ \leftarrow 1 \rightarrow \end{array} \approx \begin{array}{c} \left[\begin{array}{c} \mathbf{y}(\mathbf{w}_0) \end{array} \right] \\ \leftarrow 1 \rightarrow \end{array} + \begin{array}{c} \left[\begin{array}{c} \nabla_{\mathbf{w}} \mathbf{y}(\mathbf{w}_0)^T \end{array} \right] \\ \leftarrow p \rightarrow \end{array} \left(\begin{array}{c} \left[\begin{array}{c} \mathbf{w} \end{array} \right] \\ \leftarrow 1 \rightarrow \end{array} - \begin{array}{c} \left[\begin{array}{c} \mathbf{w}_0 \end{array} \right] \\ \leftarrow 1 \rightarrow \end{array} \right) \begin{array}{c} \updownarrow p \\ \end{array}$$

$$\phi(\mathbf{x}) = \nabla_{\mathbf{w}} f(\mathbf{x}, \mathbf{w}_0)$$

Taylor Expansion is Okay?

이론편

Weight를 initialize 하고 딱 한 번 업데이트 한다면

$$\mathbf{w}_1 = \mathbf{w}_0 - \eta \nabla_{\mathbf{w}} L(\mathbf{w}_0)$$

$$\text{net change in } \mathbf{y}(\mathbf{w}) \lesssim \|\mathbf{y}(\mathbf{w}_0) - \bar{\mathbf{y}}\|$$

$$\text{distance } d \text{ moved in } w \text{ space} \approx \frac{\text{net change in } \mathbf{y}(\mathbf{w})}{\text{rate of change of } \mathbf{y} \text{ w.r.t } \mathbf{w}} = \frac{\|\mathbf{y}(\mathbf{w}_0) - \bar{\mathbf{y}}\|}{\|\nabla_{\mathbf{w}} \mathbf{y}(\mathbf{w}_0)\|}$$

$$\text{change in model Jacobian} \approx \text{distance } d \times \text{rate of change of the Jacobian} = d \cdot \|\nabla_{\mathbf{w}}^2 \mathbf{y}(\mathbf{w}_0)\|$$

$$\text{relative change in model Jacobian} \approx \frac{d \cdot \text{rate of change of Jacobian}}{\text{norm of Jacobian}} = \frac{d \cdot \|\nabla_{\mathbf{w}}^2 \mathbf{y}(\mathbf{w}_0)\|}{\|\nabla_{\mathbf{w}} \mathbf{y}(\mathbf{w}_0)\|} = \underbrace{\|(\mathbf{y}(\mathbf{w}_0) - \bar{\mathbf{y}})\|}_{\kappa(\mathbf{w}_0)} \frac{\|\nabla_{\mathbf{w}}^2 \mathbf{y}(\mathbf{w}_0)\|}{\|\nabla_{\mathbf{w}} \mathbf{y}(\mathbf{w}_0)\|^2} \ll 1$$

Taylor Expansion is Okay?

이론편

$$\text{relative change in model Jacobian} \approx \frac{d \cdot \text{rate of change of Jacobian}}{\text{norm of Jacobian}} = \frac{d \cdot \|\nabla_w^2 \mathbf{y}(\mathbf{w}_0)\|}{\|\nabla_w \mathbf{y}(\mathbf{w}_0)\|^2} = \underbrace{\|\mathbf{y}(\mathbf{w}_0) - \bar{\mathbf{y}}\|}_{\kappa(\mathbf{w}_0)} \frac{\|\nabla_w^2 \mathbf{y}(\mathbf{w}_0)\|}{\|\nabla_w \mathbf{y}(\mathbf{w}_0)\|^2} \ll 1$$

The amount of change in $\mathbf{w}$ required to produce a change of $\|\mathbf{y}(\mathbf{w}_0) - \bar{\mathbf{y}}\|$ in $\mathbf{y}$ causes a negligible change in the Jacobian $\nabla_w \mathbf{y}(\mathbf{w})$.

y 가 $\bar{y}$ 에 가까워지도록 w 는 변하는데 그 Jacobian은 거의 변화가 없을 때
Taylor Approximation이 유의미해진다!

Taylor Expansion is Okay?

이론편

$$\|(\mathbf{y}(\mathbf{w}_0) - \bar{\mathbf{y}})\| \frac{\|\nabla_{\mathbf{w}}^2 \mathbf{y}(\mathbf{w}_0)\|}{\|\nabla_{\mathbf{w}} \mathbf{y}(\mathbf{w}_0)\|^2} \ll 1 \quad \text{그럼 어떻게 해야 } \kappa(\mathbf{w}_0) \text{ 가 작아지지?}$$

Neural Network의 output에 α 를 곱하기

$$\kappa_{\alpha}(\mathbf{w}_0) = \|(\alpha \mathbf{y}(\mathbf{w}_0) - \bar{\mathbf{y}})\| \frac{\|\nabla_{\mathbf{w}}^2 \alpha \mathbf{y}(\mathbf{w}_0)\|}{\|\nabla_{\mathbf{w}} \alpha \mathbf{y}(\mathbf{w}_0)\|^2}$$

$$\kappa_{\alpha}(\mathbf{w}_0) \sim \frac{\|\nabla_{\mathbf{w}}^2 \alpha \mathbf{y}(\mathbf{w}_0)\|}{\|\nabla_{\mathbf{w}} \alpha \mathbf{y}(\mathbf{w}_0)\|^2} = \frac{\alpha \|\nabla_{\mathbf{w}}^2 \mathbf{y}(\mathbf{w}_0)\|}{\alpha^2 \|\nabla_{\mathbf{w}} \mathbf{y}(\mathbf{w}_0)\|^2} = \frac{1}{\alpha} \frac{\|\nabla_{\mathbf{w}}^2 \mathbf{y}(\mathbf{w}_0)\|}{\|\nabla_{\mathbf{w}} \mathbf{y}(\mathbf{w}_0)\|^2}$$

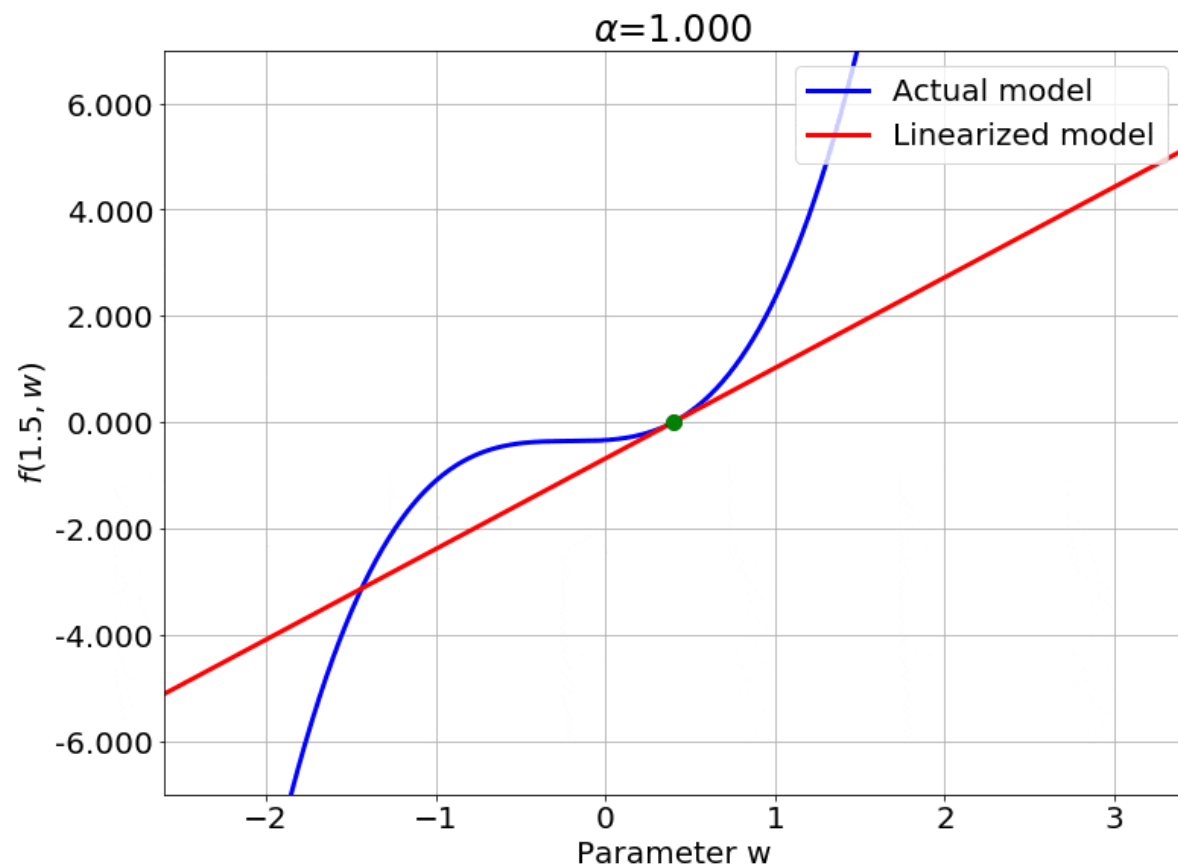
α 가 커지면 κ 는 감소

Taylor Expansion is Okay?

예시편

$$\kappa_{\alpha}(w_0) \sim \frac{\|\nabla_w^2 \alpha \mathbf{y}(w_0)\|}{\|\nabla_w \alpha \mathbf{y}(w_0)\|^2} = \frac{\alpha \|\nabla_w^2 \mathbf{y}(w_0)\|}{\alpha^2 \|\nabla_w \mathbf{y}(w_0)\|^2} = \frac{1}{\alpha} \frac{\|\nabla_w^2 \mathbf{y}(w_0)\|}{\|\nabla_w \mathbf{y}(w_0)\|^2}$$

$$f(x, w) = (wx + w^2x + \sin(e^{0.1w}))(w - 0.4)$$

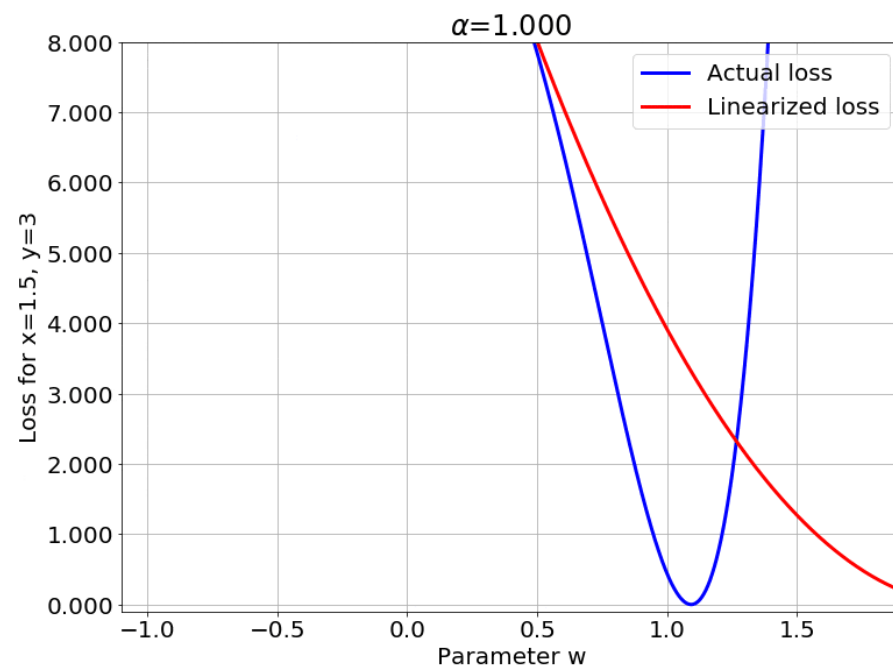


Taylor Expansion is Okay?

예시편

$$\kappa_{\alpha}(w_0) \sim \frac{\|\nabla_w^2 \alpha \mathbf{y}(w_0)\|}{\|\nabla_w \alpha \mathbf{y}(w_0)\|^2} = \frac{\alpha \|\nabla_w^2 \mathbf{y}(w_0)\|}{\alpha^2 \|\nabla_w \mathbf{y}(w_0)\|^2} = \frac{1}{\alpha} \frac{\|\nabla_w^2 \mathbf{y}(w_0)\|}{\|\nabla_w \mathbf{y}(w_0)\|^2}$$

$$f(x, w) = (wx + w^2x + \sin(e^{0.1w}))(w - 0.4)$$



α 가 커지니 loss 가 0으로 간다 => Linear 해 진다.

Taylor Expansion

정리

태초에 hidden layer가 2차까지 밖에 없는 Neural Network가 존재

Neural Network을 1차 Taylor Approximation

1차 Taylor Approximation이 Real Value와 유사하다는 거는 Linear 하다는 거네?

Taylor Approximation과 Real Value와 유사해지는 조건을 찾았다!

Q & A

Gradient Flow

이론편

$$\boldsymbol{w}_{k+1} = \boldsymbol{w}_k - \eta \nabla_{\boldsymbol{w}} L(\boldsymbol{w}_k)$$

$$\frac{\boldsymbol{w}_{k+1} - \boldsymbol{w}_k}{\eta} = -\nabla_{\boldsymbol{w}} L(\boldsymbol{w}_k)$$

$$\frac{d\boldsymbol{w}(t)}{dt} = -\nabla_{\boldsymbol{w}} L(\boldsymbol{w}(t))$$

$$\dot{\boldsymbol{w}}(t) = -\nabla L(\boldsymbol{w}(t))$$

$$L(\boldsymbol{w}) = \frac{1}{2} \|\boldsymbol{y}(\boldsymbol{w}) - \bar{\boldsymbol{y}}\|_2^2$$

$$\dot{\boldsymbol{w}} = -\nabla \boldsymbol{y}(\boldsymbol{w})(\boldsymbol{y}(\boldsymbol{w}) - \bar{\boldsymbol{y}})$$

$$\boldsymbol{y}(\boldsymbol{w}) \approx \boldsymbol{y}(\boldsymbol{w}_0) + \nabla_{\boldsymbol{w}} \boldsymbol{y}(\boldsymbol{w}_0)^T (\boldsymbol{w} - \boldsymbol{w}_0)$$

$$\dot{\boldsymbol{y}}(\boldsymbol{w}) = \nabla \boldsymbol{y}(\boldsymbol{w})^T \dot{\boldsymbol{w}} = -\nabla \boldsymbol{y}(\boldsymbol{w})^T \nabla \boldsymbol{y}(\boldsymbol{w})(\boldsymbol{y}(\boldsymbol{w}) - \bar{\boldsymbol{y}})$$

Neural Tangent Kernel! $\boldsymbol{H}(\boldsymbol{w})$

$$\phi(\mathbf{x}) = \nabla_{\mathbf{w}} f(\mathbf{x}, \mathbf{w}_0)$$

Gradient Flow

이론편

$$\mathbf{y}(\mathbf{w}) \approx y(\mathbf{w}_0) + \nabla_{\mathbf{w}} \mathbf{y}(\mathbf{w}_0)^T (\mathbf{w} - \mathbf{w}_0) \quad \dot{\mathbf{y}}(\mathbf{w}) = \nabla \mathbf{y}(\mathbf{w})^T \dot{\mathbf{w}} = -\nabla \mathbf{y}(\mathbf{w})^T \nabla \mathbf{y}(\mathbf{w}) (\mathbf{y}(\mathbf{w}) - \bar{\mathbf{y}})$$

$$\begin{aligned} \begin{matrix} \updownarrow n \\ \left[\begin{array}{c} \mathbf{H}(\mathbf{w}_0) \end{array} \right] \\ \leftarrow n \end{matrix} &= \begin{matrix} \left[\begin{array}{c} \nabla_{\mathbf{w}} \mathbf{y}(\mathbf{w}_0)^T \end{array} \right] \\ \leftarrow p \end{matrix} \begin{matrix} \left[\begin{array}{c} \nabla_{\mathbf{w}} \mathbf{y}(\mathbf{w}_0) \end{array} \right] \\ \leftarrow n \end{matrix} \begin{matrix} \updownarrow p \end{matrix} \\ \text{NTK} & \end{aligned}$$

$$= \begin{bmatrix} \text{---} \phi(\bar{\mathbf{x}}_1)^T \text{---} \\ \vdots \\ \text{---} \phi(\bar{\mathbf{x}}_n)^T \text{---} \end{bmatrix} \begin{bmatrix} | & & | \\ \phi(\bar{\mathbf{x}}_1) & \dots & \phi(\bar{\mathbf{x}}_n) \\ | & & | \end{bmatrix}$$

Gradient Flow

이론편

$\kappa(\mathbf{w}_0) \ll 1$ 인 조건이 뭐였더라?

y 가 $\bar{y}$ 에 가까워지도록 w 는 변하는데 그 **Jacobian**은 거의 변화가 없을 때
Taylor Approximation이 유의미해진다!

$$\nabla \mathbf{y}(\mathbf{w}(t)) \approx \nabla \mathbf{y}(\mathbf{w}_0) \implies \mathbf{H}(\mathbf{w}(t)) \approx \mathbf{H}(\mathbf{w}_0)$$

$$\dot{\mathbf{y}}(\mathbf{w}) = \nabla \mathbf{y}(\mathbf{w})^T \dot{\mathbf{w}} = -\nabla \mathbf{y}(\mathbf{w})^T \nabla \mathbf{y}(\mathbf{w})(\mathbf{y}(\mathbf{w}) - \bar{\mathbf{y}})$$

$$\dot{\mathbf{y}}(\mathbf{w}) = -\mathbf{H}(\mathbf{w}_0)(\mathbf{y}(\mathbf{w}) - \bar{\mathbf{y}})$$

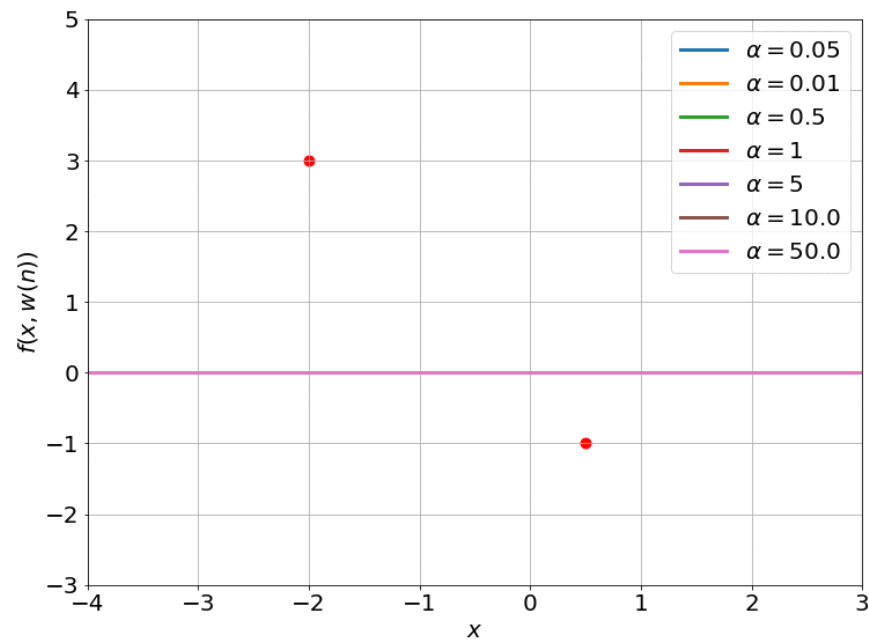
$$\dot{\mathbf{u}} = -\mathbf{H}(\mathbf{w}_0)\mathbf{u}$$

$$\mathbf{u}(t) = \mathbf{u}(0)e^{-\mathbf{H}(\mathbf{w}_0)t}$$

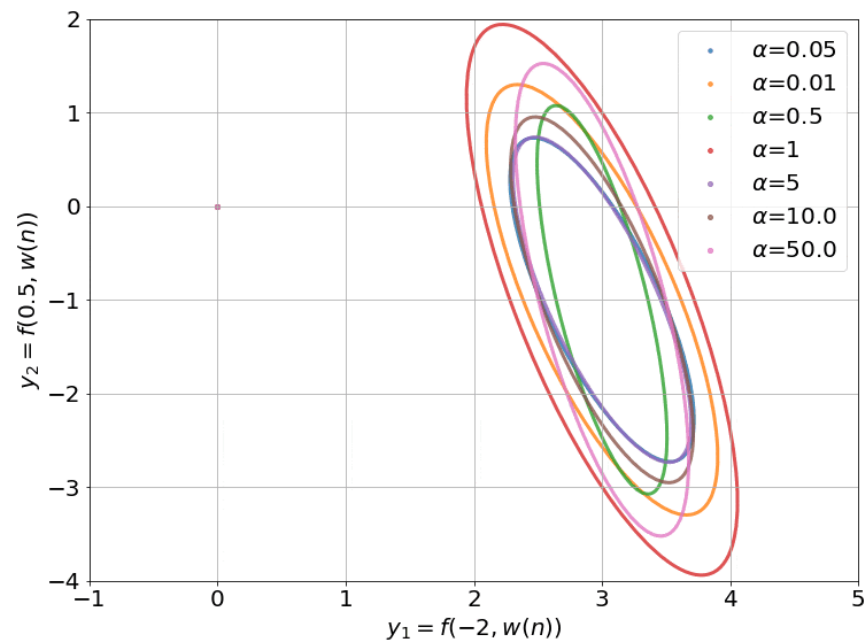
$p \gg n$ 의 경우 NTK 가 positive definite 이기 때문에 $\mathbf{u}(t)$ 는 항상 0으로 수렴함!

Gradient Flow

예시편



$(y - \bar{y})^T H(w(t))(y - \bar{y})$ 로 타원을 그렸을 때



α 가 클 때 kernel regime(kernel이 변하지 않는 것)가 성립한다!

Conclusion

정리

특정 상황에서 NN을 Linear하게 바꿔주는 NTK 라는 것을 발견

해당 조건 하에서 loss는 0으로 수렴한다는 것을 증명

MSE가 loss function일 경우 NTK에 의한 NN의 해석이 잘 된다는 것을 증명

(논문에서는) MNIST에 대한 실험적인 예시

Q & A

Thank You